



Scheduling jobs on parallel machines applying neural network and heuristic rules

Youngshin Park^{a,*}, Sooyoung Kim^a, Young-Hoon Lee^b

^a*Department of Industrial Engineering, Pohang University of Science and Technology, San 31, Hyoja, Pohang 790-784, South Korea*

^b*Department of Industrial Systems Engineering, Yonsei University, Shinchon 134, Seodaemun-gu, Seoul, South Korea*

Accepted 24 March 2000

Abstract

In this paper, we investigate the problem of scheduling jobs on identical parallel machines. The jobs are assumed to have sequence dependent setup times independent of the machine. Each job has a processing time, a due date, and a weight for penalizing tardiness. The objective of scheduling is to find a sequence of the jobs which minimizes the sum of weighted tardiness. We propose an extension of the ATCS (Apparent Tardiness Cost with Setups) rule developed by Lee et al. (1997) which utilizes some look-ahead parameters for calculation the priority index of each job. Scheduling jobs on parallel machines with sequence-dependent set-up times. *Technical report*. Columbia University; Lee, Y.H., Bhaskaran, K., & Pinedo, M., 1997. A heuristic to minimize the total weighted tardiness with sequence-dependent setups. *IIE Transactions*, 29, 45–52.) which utilizes some look-ahead parameters for calculating the priority index of each job. The look-ahead parameters were introduced as a tuning mechanism which adjusts the discount rate inside the priority calculation according to the given problem characteristics. To determine the proper values of the look-ahead parameters, Lee identified some measures for describing problem characteristics. They proposed four factors to describe properties of the problem instances, and a heuristic curve-fitting method was used to determine the equations for calculating proper values of the look-ahead parameters. In our approach, an additional factor for measuring the problem characteristics is introduced and we also utilize a neural network to get more accurate values of the look-ahead parameters. Our computational results show that the proposed approach outperforms Lee et al.'s (1997) original ATCS and a simple application of ATCS. © 2000 Elsevier Science Ltd. All rights reserved.

Keywords: Scheduling; Parallel machines; Heuristic rule; Sequence dependent setup time; Neural network

* Corresponding author. Fax: +82-562-279-2870.

E-mail addresses: pys@postech.ac.kr (Y. Park), sookim@postech.ac.kr (S. Kim), youngh@bubble.yonsei.ac.kr (Y.-H. Lee).

1. Introduction

Scheduling jobs for some identical machines is a situation frequently encountered in various manufacturing systems. As an example, in a wafer fabrication line for manufacturing semiconductor, one can find several identical machines shared by multiple device types and processes. A typical example in a wafer fab is the stepper machines capable of serving different wafer types at the photolithography process. At each stepper, when a changeover is required from one type of wafer to another, there usually exists certain setup time for changing the reticles and running test wafers.

This paper addresses the scheduling problem under such conditions: A set of jobs are waiting in a queue, identical parallel machines are available, and between the completion of a job and the start of the next one, certain setup time is required which is sequence dependent. Our scheduling objective is to find a sequence of jobs on each machine that minimizes the total tardiness multiplied by given weights. Such tardiness weights vary among the different types of jobs, which allows the scheduler to differentiate the urgency of the jobs. In the semiconductor case, one can find some wafer lots labeled as “hot” lots which are to be processed immediately on any available machine overriding the current priorities.

We assume that each job has a due date, a weight for penalizing tardiness, the processing time, and setup times different for different predecessors. It is known that the problem is NP-hard even in the case of a single machine. Thus, some heuristics have been developed to find the sequence of jobs in parallel machines for satisfying several objectives. One popular heuristic approach for the total weighted tardiness problem is using the dispatching rules which assigns a priority value to each job. The priority of a job can be calculated as a function of such job parameters as the processing time, the due date, the weight, as well as the parameters concerning the machine environment.

The Earliest Due Dates (EDD) rule and the Weighted Shortest Processing Time (WSPT) rule are the simplest. These simple rules do not generate good solutions under certain problem conditions. For example, EDD, Minimum Slack (MSLACK), and Slack per Remaining Processing Time (S/RPT) rules perform reasonably well at light load levels but deteriorate in congested shops. On the other hand, the popular rules like the Shortest Processing Time (SPT) rule fails at light load levels and generous due date allowances (Elvers & Taube, 1983).

Designed to handle the weighted tardiness problem, there are Cost Over Time (COVERT) by Carroll (1965), Apparent Tardiness Cost (ATC) rule by Vepsalainen and Morton (1987), and Apparent Tardiness Cost and Setups (ATCS) rule by Lee and Pinedo (1992) and Lee, Bhaskaran and Pinedo (1997) as more sophisticated dispatching rules. Vepsalainen and Morton (1987) compared efficiency of several priority rules in their paper to propose an ATC rule. They claimed that the rules utilizing more information outperform simpler ones: COVERT is as good as the best of the simple rules, and often much better, and their ATC rule outperforms COVERT consistently. Their rank of the rules is ATC–COVERT–WSPT–S/RPT–EDD–FCFS with the exception that S/RPT and EDD cross over WSPT in shops with generous due dates and low utilization, and EDD sometimes beats COVERT at very light load. Lee et al. (1997) developed ATCS rule, extending the ATC rule to consider the sequence-dependent setups. A quick review of the existing rules is given in the following chapter.

Kim, Lee and Agnitorti (1995) proposed application of a neural network to enhance the

performance of ATC and ATCS for single machine case. They applied a back-propagation neural network to achieve more accurate parameter estimations in calculating the job priorities. According to their tests for single machine cases, neural network could significantly enhance the performance of both ATC and ATCS. We adopt their approach, extending it to cover parallel machine situation, and also introducing additional factor for capturing problem characteristics.

Details of the proposed approach are given Section 2. In Section 3, the proposed approach is explained. Comparison results of the computational tests are presented in Section 4, then we summarize the paper and discuss the further research possibilities in Section 5.

2. Scheduling problem and research review

We consider n jobs available at time 0 and m identical machines in parallel. Job j has processing time p_j , weight w_j and due date d_j . If job j is followed by job k , a setup time s_{jk} is incurred. Such a setup time is independent of the machine and is assumed to be sequence-dependent. C_j denotes the completion time of job j , then the tardiness T_j of job j is defined as $\max(C_j - d_j, 0)$. Therefore, the objective function is $\sum w_j T_j$.

The first composite dispatching rule proposed for the total tardiness problem is COVERT rule suggested by Carroll (1965). It is primarily designed for the single and parallel machine environment, but it can also be adapted for the job shop. The priority calculation of each job under COVERT rule is shown in Table 1. In this rule, the urgency of the job increases linearly over a given range as the slack decreases. Thus, it works by dispatching overdue jobs according to the SPT rule, while jobs with large slacks have low priorities.

Vepsalainen and Morton (1987) developed an alternative rule, called (ATC) rule. They incorporated the weight w_j of job j into the dispatching rule. The priority of each job under the ATC rule at time t is shown in Table 1. The front portion of the priority of ATC represents

Table 1
The priority indices of the existing rules^a

Rule	Priority index
COVERT (Carroll, 1965)	$I_j(t) = \frac{1}{p_j} \frac{(k\bar{p} - (d_j - p_j - t)^+)^+}{k\bar{p}}$
ATC (Vepsalainen & Morton, 1987)	$I_j(t) = \frac{w_j}{p_j} \exp\left(-\frac{(d_j - p_j - t)^+}{k\bar{p}}\right)$
ATC-modified (Raman et al., 1989)	$I_j(t, l) = \frac{w_j}{p_j + s_{lj}} \exp\left(-\frac{(d_j - p_j - s_{lj} - t)^+}{k\bar{p}}\right)$
ATCS (Lee et al., 1997)	$I_j(t, l) = \frac{w_j}{p_i} \exp\left(-\frac{(d_j - p_j - t)^+}{k_1 \bar{p}}\right) \exp\left(-\frac{s_{lj}}{k_2 \bar{s}}\right)$

^a Notation: $I_j(t)$, priority index of job j ; t , current time; p_j , processing time of job j ; w_j , weight of job j ; d_j , due date of job j ; s_{lj} , sequence dependent setup time, if job j succeeds job l , s_{lj} is incurred; $\bar{p}$, average of the processing times of jobs; $\bar{s}$, average of the setup times of jobs; k , k_1 , k_2 , scaling parameter or look-ahead parameter; A^+ , $\max(A, 0)$.

WSPT rule and rear portion represents that jobs with large slacks have low priorities. Therefore, ATC rule is basically a mixture of the WSPT rule and the Least Slack Remaining (LSR) rule. In ATC rule, however, the effect of slack on priority decreases exponentially as slack increases.

Ow and Morton (1989) extended the ATC rule to the problem which assumes a penalty for earliness as well as for tardiness. This rule was applied to a single machine, parallel machines, as well as some other machine environment. Morton, Lawrence, Rajagopalan and Kekre (1988) developed the scheduling module, SCHED-STAR, which uses cost–benefit analysis to make scheduling decisions with a price-based objective function, in which the weighted tardiness is considered as part of total cost.

Raman, Rachamadugu and Talbot (1989) suggested a modification of the ATC rule to take setup times into account. The priority formula for ATCS-modified in Table 1 includes sequence dependent setup times. A setup time s_{jl} is incurred when job j is followed by job l . The priority indices of all remaining jobs are computed and the job with the highest index is scheduled to go next. This rule essentially modifies the ATC rule by replacing the processing time of a job with the sum of the processing time and the required setup.

Lee and Pinedo (1992) and Lee et al., (1997) developed ATCS rule, which considers setup time in addition to ATC rule. The priority calculation of ATCS rule is also shown in Table 1. The ATCS, in contrast to Raman's rule, separates the effect of the remaining slack from the effect of the setup time. The priority of a job given by the WSPT ratio is exponentially discounted twice, once based on slack and again based on setup. These two effects are scaled separately by the parameters k_1 and k_2 , which jointly provide fine-tuning capability for the ATCS rule. These two parameters are called look-ahead parameters or scaling parameters and are defined by the properties of the problem instances. The ATCS rule is described in detail later.

3. Proposed approach

Before we present our extension of Lee et al.'s (1997) ATCS rule, we first give a quick review of ATCS.

3.1. Lee et al.'s ATCS rule

Lee et al.'s (1997) ATCS consists of two stages. In the first stage, more like a pre-processing step, they run some statistical analysis of the problem instances randomly generated, and define formulas to estimate makespan and four factors to characterize a problem instance. In the second stage, using these estimates, they calculate the values of the look-ahead parameters to be used in applying the ATCS priority index for a given problem. The sequence of the jobs is determined by assigning jobs to the machines available at the earliest time point according to the priority indices.

Four factors used to quantify the problem characteristics are: due date tightness, due date range, setup time severity, and job–machine factor. The first factor is the due date tightness, τ , defined as

$$\tau = 1 - \frac{\bar{d}}{C_{\max}}$$

where $\bar{d}$ is the average of the due dates and $C_{\max}$ is the makespan which is the completion time of the last job to leave the system. The value of τ ranges over $[0, 1]$ under the assumption that due dates are greater than 0 and less than the makespan. The larger the value of τ is, the tighter the schedule is. Obviously, the makespan is schedule-dependent, and therefore, difficult to determine in advance before the jobs are scheduled. An estimator of makespan is necessary and described later.

The second factor is the due date range factor, which is defined as

$$R = \frac{(d_{\max} - d_{\min})}{C_{\max}}$$

where $d_{\max}$ or $d_{\min}$ is the maximum or minimum value of the due dates. While τ gives an indication of the tightness of the due date, R , on the other hand, provides a rough measure of the spread of the due dates.

The third factor is the setup time severity factor, which is defined as

$$\eta = \frac{\bar{s}}{\bar{p}}$$

where $\bar{s}$ is the average of all setup time combination, i.e., $\bar{s} = \sum_i \sum_j \frac{s_{ij}}{n(n+1)}$ and $\bar{p}$ is the average of all processing time. It represents the relative significance of the setup times compared to the processing times.

The fourth factor which plays an important role in the parallel machine environment is so-called job-machine factor:

$$\mu = \frac{n}{m}$$

where n is the number of jobs and m is the number of machines in parallel. The factor μ describes the average number of jobs to be processed on one machine.

To estimate the makespan before any sequence of the jobs is defined, Lee et al. (1997) proposed the following equation:

$$C_{\max} = (\beta \bar{s} + \bar{p})\mu$$

where β is a coefficient, which takes into account the effect of the setup times on the makespan and μ is job-machine factor. They suggested the following function to estimate β for μ sufficiently large (e.g. $\mu \geq 5$):

$$\beta = 0.40 + 10/\mu^2 - \eta/7$$

It is assumed that every machine is available at time zero. A schedule for the parallel machines is determined as follows: whenever one machine becomes free at time t (initially 0), compute the following index for the set of unscheduled jobs, if any,

$$I_j(t, l) = \frac{w_j}{p_j} \exp \left(- \frac{(d_j - p_j - t)^+}{k_1 \bar{p}} \right) \exp \left(- \frac{s_{lj}}{k_2 \bar{s}} \right)$$

where l is the last job completed on the machine which just has become free. At time t , the job with the largest index is selected. If more than one machine is free at one time, choose one arbitrarily. The look-ahead parameters, k_1 and k_2 , are determined by applying the following functions:

$$k_1 = 1.2 \ln(\mu) - R,$$

$$k_2 = \frac{\tau}{A_2 \sqrt{\eta}}$$

where $A_2 = 1.8$ if $\tau < 0.8$, $A_2 = 2$ if $\tau \geq 0.8$. The following modification has to be made:

1. subtract 0.5 from k_1 if $\tau < 0.5$,
2. subtract 0.5 from k_2 if $\eta < 0.5$ and $\mu > 5$.

These functions of scaling parameters and factors of problem are derived from simulation experiments and heuristic curve fittings. More details can be found in Lee et al. (1997).

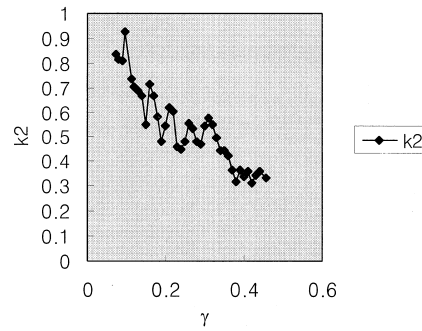
3.2. Proposed extensions

The differences between Lee et al.'s (1997) approach and that of ours in this paper are: First, we introduce an additional factor to describe the characteristics of the problems. Secondly, a neural network is trained through simulated problem sets, and used to generate the values of the look-ahead parameters. In addition to four factors used in Lee et al.'s (1997) approach, we introduce a new factor termed *setup time range* to find the better estimate of k_1 and k_2 . The setup time range, γ , is defined as

$$\gamma = \frac{S_{\max} - S_{\min}}{P_{\max}}$$

where $S_{\max}$ and $S_{\min}$ denote the maximum and the minimum of averages of each job's setup times, respectively. $P_{\max}$ is the maximum value of the processing times. Dividing the difference of setup times by the maximum value of processing time is for normalization. We expect that γ affects k_2 rather than k_1 because k_2 is the parameter used while considering the setups in calculating the priorities.

In order to test the relationship between γ and k_2 , we solved many random samples, and then obtained the values of the scaling parameters that achieve the minimum objective value. The procedure of the simulation tests is detailed in the computational tests later. Fig. 1 shows the resulting relationship between γ and k_2 . As γ grows, 'correct' value of k_2 becomes small. As k_2 decreases, the effect of setup time portion in the priority index increases. That is, if the range of setup time is larger, more consideration of the setup times is to be forced in the priority calculation. Due to this correlation, we can expect better results by adding γ to the existing factors.

Fig. 1. Relationship between γ and k_2 .

3.3. Application of neural network

In order to determine the values of the scaling parameters k_1 and k_2 , application of neural networks is proposed in this paper in lieu of heuristic curve-fitting proposed by Lee et al. (1997). A back-propagation neural network is adopted for our research, in which signals are passed forward from the input layer to the output layer through a hidden layer, and the learning is done by adjusting the connection weights by using the back-propagation algorithm. We used the network which has a single hidden layer with seven nodes and bias as depicted in Fig. 2. The number of input nodes is five for the factors τ , R , η , μ , γ , and the number of output node is one for the parameters k_1 or k_2 .

Hyperbolic tangent function is used as transfer function at each node of network. The number of nodes in the hidden layer was determined by testing several alternative configurations of the network model. To our knowledge, no theoretical finding is available for optimal design of such a network. An explanation about training set is provided in the following section.

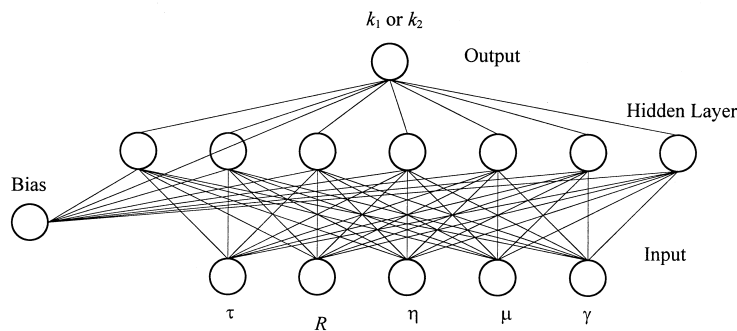


Fig. 2. Neural network with single hidden layer.

4. Computational test

4.1. Experimental design

In this section, we describe experiments performed to make training set for neural network and methods to compare our neural net-based ATCS with the conventional rules. The number of jobs is set to 60 throughout the study. We generate 4032 different combinations of factors, τ , R , η , μ and γ , as follows:

$$t = (0.3, 0.4, 0.5, \dots, 0.9)$$

$$R = (0.25, 0.5, 0.75, 1.0)$$

$$\eta = (0.25, 0.5, 0.75, 1.0)$$

$$\mu = (5, 10, 15, 20, 30, 60)$$

$$\gamma = (0.05, 0.1, 0.15, 0.2, 0.25, 0.3)$$

The processing time p_j of job j is generated from the uniform distribution over the interval $[50, 150]$ with the mean, $\bar{p}$, 100. The due date d_j is generated using two factors τ and R , which are related to due date. The average of due date $\bar{d}$ is calculated from τ , then the due date is uniformly distributed over the interval $[(1 - R)\bar{d}, \bar{d}]$ with probability τ and uniformly distributed over the interval $[\bar{d}, \bar{d} + (C_{\max} - \bar{d})R]$ with probability $(1 - \tau)$. The average of the due dates generated is $\bar{d}$. The weight w_j of job j is generated from the uniform distribution on $[0, 10]$. Generation of the setup time is controlled by η . The setup time considered in this paper is sequence dependent, therefore, each job has first setup time and setup times dependent upon the followers. These setup times are uniformly distributed over the interval $[0, 2\eta\bar{p}]$. For 40320 instances, 10 different instances for 4032 different types, our ATCS rule is applied to different values of k_1 and k_2 :

$$k_1 = (0.2, 0.4, 0.6, \dots, 6.4)$$

$$k_2 = (0.1, 0.2, 0.3, \dots, 1.6)$$

For each combination of k_1 and k_2 , we solved the problem, then the value of objective function, i.e., the sum of the weighted tardiness, is computed. All k_1 and k_2 values which lead to minimum values of the objective function are identified and the average is taken for both k_1 and k_2 . For each combination of factors, there are 10 averages of k_1 and k_2 , respectively which are computed above. The average of these averages is taken for the optimal value for each problem. The instance set with the optimal values is used for training of the neural network.

In the following, we compare the tardiness values of four different approaches as below and present the improvement of our extended ATCS rule over other rules.

‘fixed’ using fixed k_1 and k_2 for computing index.

- ‘Lee’ using Lee et al.’s (1997) function of k_1 and k_2 for computing index.
- ‘Neu’ applying neural network with four input factors to get the value of scaling parameters and using the outputs for computing index.
- ‘Neu_G’ applying neural network with five input factors included γ to get the value of scaling parameters and using the outputs for computing index.

The first case is using the same scaling parameter for all problems, i.e., no adjustment of the parameter for the problems. In this case, we simply use averages of scaling parameter, which are computed from the training set of the neural network. The second case uses the function of scaling parameter and factors by Lee et al. (1997) in order to get k_1 and k_2 . The third case uses neural network with four input nodes except γ for getting values of k_1 and k_2 . The last case is our main approach, using neural network with five inputs including γ , to get the estimates of the scaling parameters.

4.2. Simulation results

In the simulation tests, we generated 4032 random samples for the combination of factors and compared the tardiness value of above four cases. The method of computing improvement between the two cases is described as follows:

$$\text{Improvement (\%)} = \frac{(\text{worse rule}) - (\text{better rule})}{(\text{worse result of two})} \times 100$$

where the value inside parenthesis is the value of objective function. Table 2 summarizes the results of comparison of several cases.

Average improvement in the last column of Table 2 means the average of the improvements of 4032 problems calculated by above formula. Our main interest in Table 2 is the comparison between ‘Neu_G’, ‘Neu’ and ‘Lee’. Average improvement of ‘Neu_G’ over ‘Lee’ and ‘Neu’ over ‘Lee’ is 6% and 4%, respectively. These results indicate that using the neural network is more efficient than Lee et al.’s (1997) curve fitting approach. Average improvement of ‘Neu_G’ over ‘Neu’ is 2%, and these improvement results from adding the new characterization factor γ . Therefore, our expectation earlier — range of setup time affects k_2 and the objective function — is proven to be correct. Our approach gains 4% improvement by using the neural network and gains 2% improvement by using the additional factor γ .

Tables 3–7 show the average improvements of the cases for different ranges of each factor. The reason for considering factors divided into ranges is to see the efficiency of the rules for different properties of the problems.

Table 3 shows a trend of improvement which has the highest point in the middle value of the due date tightness comparing ‘fixed’ or ‘Lee’ with ‘Neu’ or ‘Neu_G’. This trend means, using neural network has its efficiency when the due date tightness of the jobs is appropriate. As τ increases, the difference between ‘Neu’ and ‘Neu_G’ becomes small. Because the difference between ‘Neu’ and ‘Neu_G’ is due to the effect of γ , γ does not affect the result when the due date tightness gets close to 1.

In Table 4, the improvement has its peak in the middle of the due date range value like the

Table 2
Comparison of the rules

Rules	Number of better cases	Percentage of better cases (%)	Number of worse cases	Percentage of worse cases (%)	Average improvement (%)
Neu_G vs. Lee	2049	51	1268	31	6
Neu vs. Lee	1882	47	1437	36	4
Neu_G vs. Neu	1725	43	1187	29	2
Neu_G vs. fixed	2560	63	1022	25	13
Neu vs. fixed	2469	61	1105	27	11
Lee vs. fixed	2285	57	1308	32	7

case of τ . This tells us that our approach shows higher efficiency when the due dates of jobs are spread neither too wide nor too narrow.

Factor η is setup time severity which represents the ratio of average of setup time versus average of processing time. The improvement of using neural network over heuristic curve fitting method is large at low value of η , which implies that the improvement by factor γ is not much affected by η .

However, the improvement of using neural network or heuristic curve fitting method over the method of ‘fixed’ is large at high value of η . This trend is found definitely in Fig. 3. Because we used the value of scaling parameters in the ‘fixed’ case as the averages of all cases in training set for neural network, we can predict that the higher improvement is due to low efficiency of ‘fixed’ in high value of η .

μ is the information about the average number of jobs processed in each machine. Therefore,

Table 3
Average improvement for different values of τ

τ	Neu_G over Lee (%)	Neu over Lee (%)	Neu_G over Neu (%)	Neu_G over fixed (%)	Neu over fixed (%)	Lee over fixed (%)
0.3	−3	−9	7	17	11	19
0.4	7	4	4	25	22	18
0.5	26	24	4	27	24	0
0.6	11	10	2	15	14	5
0.7	1	1	0	5	5	4
0.8	0	0	0	2	2	1
0.9	0	0	0	1	0	1

Table 4
Average improvement for different values of R

Range	Neu_G over Lee (%)	Neu over Lee (%)	Neu_G over Neu (%)	Neu_G over fixed (%)	Neu over fixed (%)	Lee over fixed (%)
0.25	5	5	0	14	14	9
0.5	9	6	4	20	17	11
0.75	8	4	5	15	12	7
1	3	2	1	3	2	0

Table 5
Average improvement for different values of η

η	Neu_G over Lee (%)	Neu over Lee (%)	Neu_G over Neu (%)	Neu_G over fixed (%)	Neu over fixed (%)	Lee over fixed (%)
0.25	10	8	3	11	9	2
0.5	9	8	1	8	7	−1
0.75	4	1	4	15	12	11
1	2	0	2	18	17	16

as μ increases, the number of machines in this system decreases and μ equal to 60 means single machine system because the total number of jobs is 60. When there is a single machine in the system, the average improvements of ‘Neu_G’ over ‘Lee’ and ‘Neu’ over ‘Lee’ are minus. In order to improve the result of single machine case, we may use neural network separately from multi machine case. When μ is small, that is, the number of machines is relatively large, there is bigger improvement.

As γ varies, there is no sudden change of improvement in all cases, but it shows a trend similar to η case.

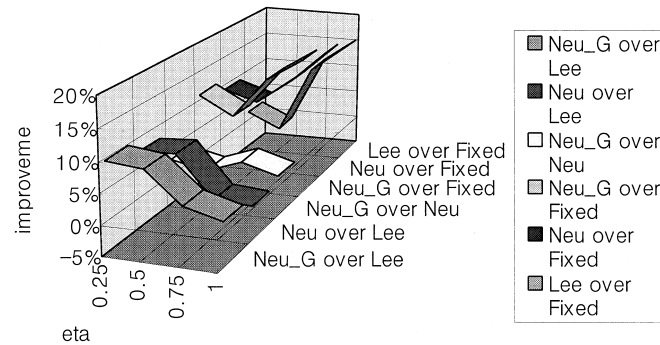
Table 6
Average improvement for different values of μ

μ	Neu_G over Lee (%)	Neu over Lee (%)	Neu_G over Neu (%)	Neu_G over fixed (%)	Neu over fixed (%)	Lee over fixed (%)
5	8	7	2	19	19	14
10	11	8	4	16	13	6
15	9	8	2	11	9	1
20	7	5	2	8	6	0
30	4	0	4	8	5	3
60	−2	−2	1	15	15	17

Table 7

Average improvement for different values of γ

γ	Neu_G over Lee (%)	Neu over Lee (%)	Neu_G over Neu (%)	Neu_G over fixed (%)	Neu over fixed (%)	Lee over fixed (%)
0.12387	8	7	2	10	9	2
0.19092	8	6	3	12	9	4
0.23494	5	3	3	12	10	6
0.27533	6	4	2	14	13	8
0.31991	4	3	2	14	12	9
0.38768	5	2	4	16	14	11

Fig. 3. Average improvement over different range of η .

In general, using neural network has its efficiency in the middle values of τ and R , and in the low values of η , μ , and γ . Finally, Fig. 4 shows the average improvement of 'Neu_G' over 'Lee' with respect to τ and μ . The improvement pattern varies with the value of τ , but has a similar form with respect to μ except μ equal to 60. This says that the efficiency of our approach is varied more with the due date of jobs rather than with the number of machines.

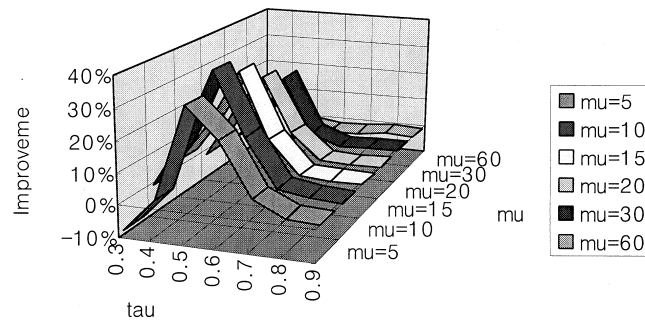


Fig. 4. Average improvement of 'Neu_G' over 'Lee'.

5. Summary and further research

We proposed an approach for solving a parallel-machine scheduling problem applying a known heuristic rule combining it with a neural network technique. Our approach is an extension of so-called ATCS rule, adding a new factor for characterizing the problem and applying a back-propagation neural network for getting the right values of the scaling parameters. The new factor, setup time range named γ , is shown to have some correlation with the parameter k_2 . After trained by a sample problem set, the neural network generates the necessary estimates of the parameters which are fed to the priority rule.

Through computational tests, it is shown that addition of the new factor improved the result by average of 2%. Applying the neural network instead of the heuristic curve fitting resulted in the average improvement of 4% over the original ATCS rule. As a result, our approach improved the objective by average 6% over Lee et al.'s (1997) ATCS rule.

While the neural network shows better results in estimating the necessary parameter values, some caution must be taken when one applies it, due to the inherent drawbacks of the neural networks. First, the neural network needs a training data set with sufficient sample cases. Sample cases with 'good' solutions may be generated through some simulations of the given problem, but sometimes it may take considerable computing time. Thus, when it is too costly or time-consuming to generate such a training set, one may have to rely on the simpler method for estimating the parameters. Second, the neural network does not provide the information how it arrived at the particular solution. However, if one needs to know how the neural network behaves for the input values, it is recommended to plot the outcomes over the ranges of the input values and 'see' the pattern of the generated surfaces. The 'black-box' nature of the neural network needs some careful application of it in many situations.

There may be many different research possibilities extending the proposed approach here. One possibility is applying a neural network to capture the problem characteristics prior to using one to estimate the parameters. Therefore, the first neural network generates the values of the problem characteristics, and then the second one uses the results as input values to estimate the parameter values. The proposed approach may be applied to different types of scheduling problems, including dispatching rules for the dynamic environment.

References

- Carroll, D. C. (1965). *Heuristic sequencing of jobs with single and multiple components*. Ph.D. thesis. Sloan School of Management, MIT.
- Elvers, D. A., & Taube, L. R. (1983). Time completion for various dispatching rules in job shops. *OMEGA*, 11(1), 81–89.
- Kim, S. Y., Lee, Y. H., & Agnihotri, D. (1995). A hybrid approach to sequencing jobs using heuristic rules and neural network. *Production Planning and Control*, 6(5), 445–454.
- Lee, Y. H., Bhaskaran, K., & Pinedo, M. (1997). A heuristic to minimize the total weighted tardiness with sequence-dependent setups. *IIE Transactions*, 29, 45–52.
- Lee, Y. H., & Pinedo, M. (1992). Scheduling jobs on parallel machines with sequence-dependent setup times. *Technical report*, Columbia University.
- Morton, T., Lawrence, S. R., Rajagopalan, S., & Kekre, S. (1988). SCHED-STAR: a price-based shop scheduling module. *Journal of Manufacturing and Operations Management*, 1, 131–181.

- Raman, N., Rachamadugu, R. V., & Talbot, F. B. (1989). Real-time scheduling of an automated manufacturing center. *European Journal of Operations Research*, 40, 222–242.
- Ow, P. S., & Morton, T. (1989). The single machine early/tardy problems. *Management Science*, 35, 177–191.
- Vepsäläinen, A., & Morton, T. (1987). Priority rules for job shops with weighted tardiness costs. *Management Science*, 33, 1035–1047.